

Alumnos: Ascurra Heber, Ditlevsen Axel, Arreceygor Lucas, Guajardo Matias Jesus Gabriel, Villafuerte Marcos Gaston.

CLASE 6: LINUX

A) Conceptos Clave (Pag 7)

- 1: Kernel: Núcleo que comunica el hardware con el software.
- 2: GPL: Licencia legal que garantiza las libertades del usuario.
- 3: Código Abierto: Software cuyo código fuente es publico y colaborativo.
- 4: Software libre: Software que respeta la libertad de uso y modificacion.

B) Comprobando conocimientos (Pag 12)

¿Que componente de GNU/LINUX fue creado por Linus Torvalds?

1. Licencia GPL

2. El Kernel (Núcleo)

3. La suite LibreOffice

4. El entorno gráfico GNOME

RTA: En 1991, el estudiante finlandés Linux Torvalds creó el componente vital llamado KERNEL (núcleo) que actúa como puente entre el hardware (procesador, memoria, etc) y tus programas.

C) Si el software es libre, ¿cómo se mantiene y mejora? ¿Por qué miles de programadores trabajan gratis en ellos? (Pag 13)

1. ¿Cómo se mantiene y mejora si es libre?

El software libre no sobrevive por "magia", sino a través de un ecosistema colaborativo muy bien organizado.

- Soporte Corporativo: Empresas gigantes (como Google, Red Hat, IBM o Microsoft) utilizan software libre para sus propias infraestructuras. Les sale más barato pagar a programadores para que mejoren un proyecto existente (como Linux) que crear uno desde cero.
- Fundaciones sin fines de lucro: Organizaciones como la Linux Foundation, la Apache Software Foundation o la Mozilla Foundation recaudan donaciones y gestionan fondos para pagar servidores, abogados y desarrolladores de tiempo completo.
- Ciclo de Retroalimentación: Al ser código abierto, cualquier usuario que detecte un error puede corregirlo y proponer la mejora. Esto crea un ciclo de mantenimiento constante que el software privado (donde solo los empleados de la empresa pueden ver el código) no siempre logra igualar en velocidad.

2. ¿Por qué miles de programadores trabajan "gratis"?

Aquí es donde entra el factor humano y profesional. La realidad es que muchos no trabajan gratis, y los que sí lo hacen, suelen obtener beneficios indirectos.

Razones principales:

Sueldo Corporativo: La mayoría de las contribuciones a grandes proyectos de software libre son hechas por programadores a los que sus empresas les pagan para hacer precisamente eso.

Prestigio y CV: Contribuir a un proyecto famoso es la mejor carta de presentación. Un perfil de GitHub con aportes en software libre vale más que cualquier título para muchos reclutadores.

Necesidad propia: Un programador usa una herramienta y descubre que le falta una función. La desarrolla para su propio beneficio y luego la comparte para que la comunidad la mantenga.

Aprendizaje y Mentoría: Trabajar en estos proyectos permite a los novatos aprender de los mejores ingenieros del mundo mediante la revisión de código (code review).

CLASE 6: GIT

Control de comando (Pag 5): ¿que comando usarias para ver la lista de todas las ramas existentes en tu proyecto?:

1. Git log
2. Git Branch
3. Git Checkout
4. Git Merge

RTA: Usamos git branch

Repaso de comandos (pag 12):

- 1: Merge = unir una rama temporal a la rama principal.
- 2: Log = visualizar el historial cronológico del proyectos
- 3: Checkout = Cambiar de una rama a la otra
- 4: Stash = guardar cambios temporalmente para atender una urgencia.

Desafio Final: Escenario real (Pag 13):

Estás terminando una nueva funcionalidad en la rama 'login-fix' y tu jefe te pide un cambio urgente en la pagina de inicio que debe subirse YA. ¿como procederias paso a paso usando git para no perder tu trabajo actual?

En esa situación, este es el flujo exacto que tendríamos que hacer para cumplir con el pedido del jefe sin perder ni un bit de lo que ya avanzamos en la funcionalidad.

1. Poner a resguardo el trabajo actual

Como no hemos terminado los cambios en “login-fix” , no queremos hacer un commit incompleto. Por eso, guardamos todo en un “espacio temporal”: `git stash`

2. Volver a la rama principal

Tenemos que movernos a la rama estable (usualmente main -o- master) para asegurarnos de que el arreglo salga desde la versión que ya está en producción: `git checkout main`

3. Actualizar el repositorio local

Antes de tocar nada, nos traemos los últimos cambios que otros compañeros hayan subido para evitar conflictos: `git pull origin main`

4. Crear una rama específica para la urgencia

Para trabajar de forma limpia y aislada, crearemos una nueva rama de "hotfix": `git checkout -b fix-home-urgente`

5. Realizar el cambio y guardarlo

Hacemos la edición en el código de la página de inicio, preparamos el archivo y registramos el cambio: `git add .` `git commit -m "Corrección urgente en el inicio"`

6. Integrar el arreglo en la rama principal

Regresamos a la rama principal y fusionamos el arreglo que acabamos de terminar: `git checkout main`, `git merge fix-home-urgente`

7. Subir el cambio inmediatamente

Envíamos la corrección al servidor para que el cambio impacte "YA", como pidió el jefe: `git push origin main`

8. Retomamos la tarea original

Ahora que la urgencia terminó, volveremos a donde estábamos trabajando antes: `git checkout login-fix`

9. Recuperar los cambios del “espacio temporal”

Traemos de vuelta todo el código que guardamos en el paso 1 para seguir trabajando exactamente donde nos quedamos: `git stash pop`